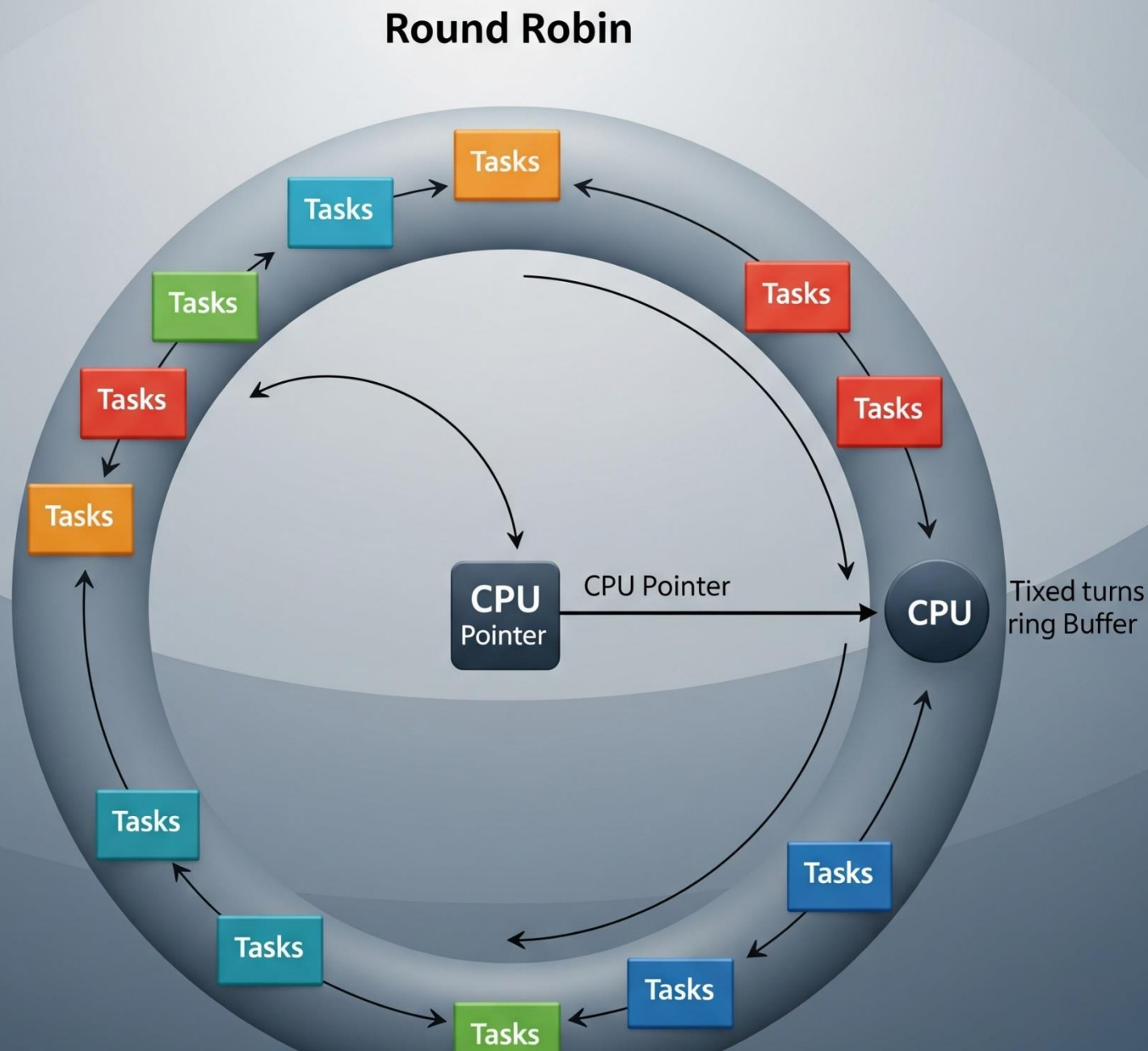


round robin algorithm



Teacher: Mohammed Al-Musa'idin

ناریز > 08

Student: Hassan Al-Omari

University number: 320220603072

Introduction

The basic idea of the Round Robin algorithm (abbreviated as RR) is to allocate short time periods (Time Quantum) to each process in a circular order (circular in the form of a list). When the time allocated to a process ends without it being completed, the process is returned to the end of the queue (Ready Queue) and the processing unit moves on to executing the next process in the cycle.

(RR) PROPERTIES

- 1. Fairness:**
Ensuring that each process gets an equal share of processing time in each cycle.
- 2. Response Time:**
Less than others.
- 3. Starvation Prevention**
Reduce.
- 4. Ease of implementation.**
- 5. High Interactivity.**

(RR) FEATURES

- 1. Ease of understanding and application.**
- 2. Reduce average waiting time.**
- 3. Starvation-Free.**
- 4. Good Responsiveness.**
- 5. Flexible and easy to adjust.**

DISADVANTAGES (RR)

- 1. Effect of Time Quantum size on performance:** If the quantum is too small: context switch rates increase, if the quantum is too large: behavior approaches FCFS algorithm.
- 2. Priority Ignorance.**
- 3. Waiting time may be higher on average for some loads.**

Implementation details: Enter the process names, enter the quantum number, and enter the remaining burst time like this: `avg(remBt, q, wt, process)`.

The programming language is Python.

I did not use any tools.

Function waiting_time(remBt, q, waitTime, completeTime, p, g):

remBt: List of remaining burst times for each process
q: Quota time
waitTime: List to be populated with waiting times for each process
completeTime: List to be populated with completion times for each process
p: List of process IDs in the same order as remBt
g: Initially, an empty list, to which we will add the process ID for each time cycle to build the Gantt chart

1. Create a copy of the remaining burst times (rBt) for later calculations:

rBt $\leftarrow$ Copy the contents of remBt

2. Set the variable time = 0 # to represent the current total time since the start of the schedule

3. Repeat until all processes have finished:

repeat:

final $\leftarrow$ true # Flag indicating that all processes have finished (will be reflected (Later, if there is work remaining)

Review each process i

For each process i from 0 to (number of processes – 1):

If (remBt[i] > 0): # If work i has non-zero execution time remaining, it has not finished yet

final $\leftarrow$ false # At least one process has not finished yet

If (remBt[i] > q): # If work has more time remaining than the quota q:

time $\leftarrow$ time + q # Execute the work in q time units

remBt[i] $\leftarrow$ remBt[i] – q # Decrease the remaining time for the work

Add p[i] to the list g # Record the running process in the Gantt chart

Else: # If work has time remaining $\leq$ q, it will finish in this iteration

time $\leftarrow$ time + remBt[i] # Execute the work for the remainder of its duration

completeTime[i] $\leftarrow$ time # Completion time for work i

waitTime[i] $\leftarrow$ time – rBt[i] # Wait time = (Completion time – Original execution time)

remBt[i] $\leftarrow$ 0 # Remaining execution time becomes zero

Add p[i] to list g # Record the running process in the Gantt chart

if (final == true): # If no process is found, remBt[i] > 0

Exit the loop (all processes have finished)

Test results

Case 1

```
remBt=[7,4,9]
process = ['p1','p2','p3']
q = 3
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

Completion Time: [17, 13, 20]
Turnaround Time: [17, 13, 20]
Waiting Time: [10, 9, 11]
Average Turnaround Time: 16.666666666666668
Average Waiting Time: 10.0
```

Case 2

```
remBt=[7,9,4]
process = ['p1','p2','p3']
q = 3
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

Completion Time: [17, 20, 16]
Turnaround Time: [17, 20, 16]
Waiting Time: [10, 11, 12]
Average Turnaround Time: 17.666666666666668
Average Waiting Time: 11.0
```

Case 3

```
remBt=[4,7,9]
process = ['p1','p2','p3']
q = 3
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

Completion Time: [10, 17, 20]
Turnaround Time: [10, 17, 20]
Waiting Time: [6, 10, 11]
Average Turnaround Time: 15.666666666666666
Average Waiting Time: 9.0
```

Case 4

```
remBt=[7,4,9]
process = ['p1','p2','p3']
q = 9
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

Completion Time: [7, 11, 20]
Turnaround Time: [7, 11, 20]
Waiting Time: [0, 7, 11]
Average Turnaround Time: 12.666666666666666
Average Waiting Time: 6.0
```

Case 1

Gantt chart: p1 --> p2 --> p3 --> p1 --> p2 --> p3 --> p1 --> p3

Case 2

Gantt chart: p1 --> p2 --> p3 --> p1 --> p2 --> p3 --> p1 --> p2

Case 3

Gantt chart: p1 --> p2 --> p3 --> p1 --> p2 --> p3 --> p2 --> p3

Case 4

Gantt chart: p1 --> p2 --> p3

Here, we notice that in **cases 1 through 3**, we only change the order of the remaining burst time list, which changes the average waiting time and average turnaround time. We also notice that the values decrease as we sort the remaining burst time list, placing the smallest values first, and increase as we sort the list, placing the largest values first. This **is because** small values complete their work quickly, while large values take time to complete their work.

We also notice that when we change **q** to a larger value, the algorithm works more like **FCFS**.

Conclusion and Improve

As we've seen, the order of operations clearly affects the results. I made a simple **improvement** to the algorithm by **ordering the operations before entering** them into the algorithm.

As we've observed that the value of **q** affects the results, I filled it with the **average of the remaining burst time**.

The algorithm now performs better than before, significantly **reducing context switching**.

Before



After

```
remBt=[9,7,4]
process = ['p1','p2','p3']
q = 3
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

Completion Time: [19, 20, 16]
Turnaround Time: [19, 20, 16]
Waiting Time: [10, 13, 12]
Average Turnaround Time: 18.333333333333332
Average Waiting Time: 11.666666666666666
```

```
remBt=[9,7,4]
process = ['p1','p2','p3']
sortp(process, remBt)
q = meanRemBT(remBt)
wt = [0]*len(remBt)
avg(remBt,q,wt,process)

[4, 7, 9]
['p3', 'p2', 'p1']
Completion Time: [4, 17, 20]
Turnaround Time: [4, 17, 20]
Waiting Time: [0, 10, 11]
Average Turnaround Time: 13.666666666666666
Average Waiting Time: 7.0
```

Before

Gantt chart: p1 --> p2 --> p3 --> p1 --> p2 --> p3 --> p1 --> p2



After

Gantt chart: p3 --> p2 --> p1 --> p2 --> p1